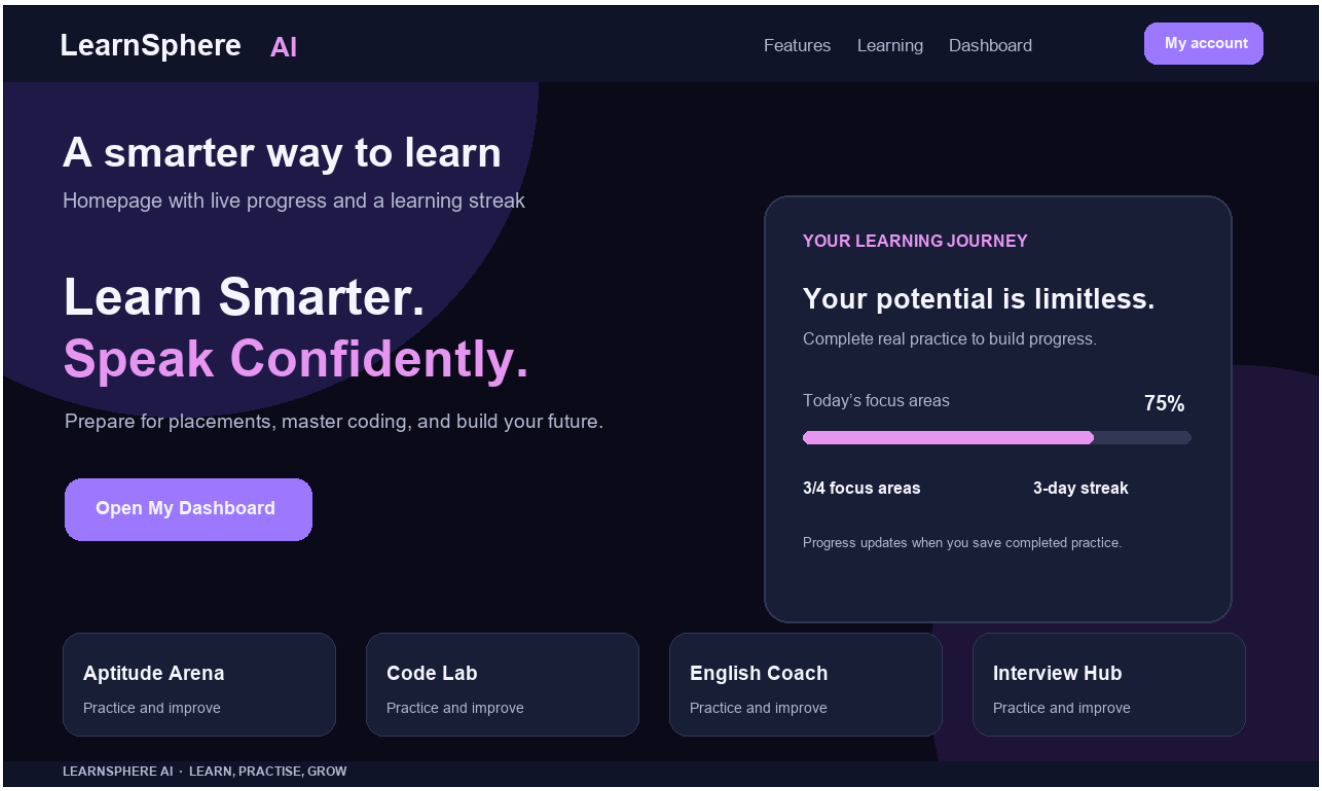
Project guide

A practical tour of the learning platform, its progress tracking, and the live Render deployment.



Illustrative signed-in state: 3 of 4 areas completed and a 3-day streak.

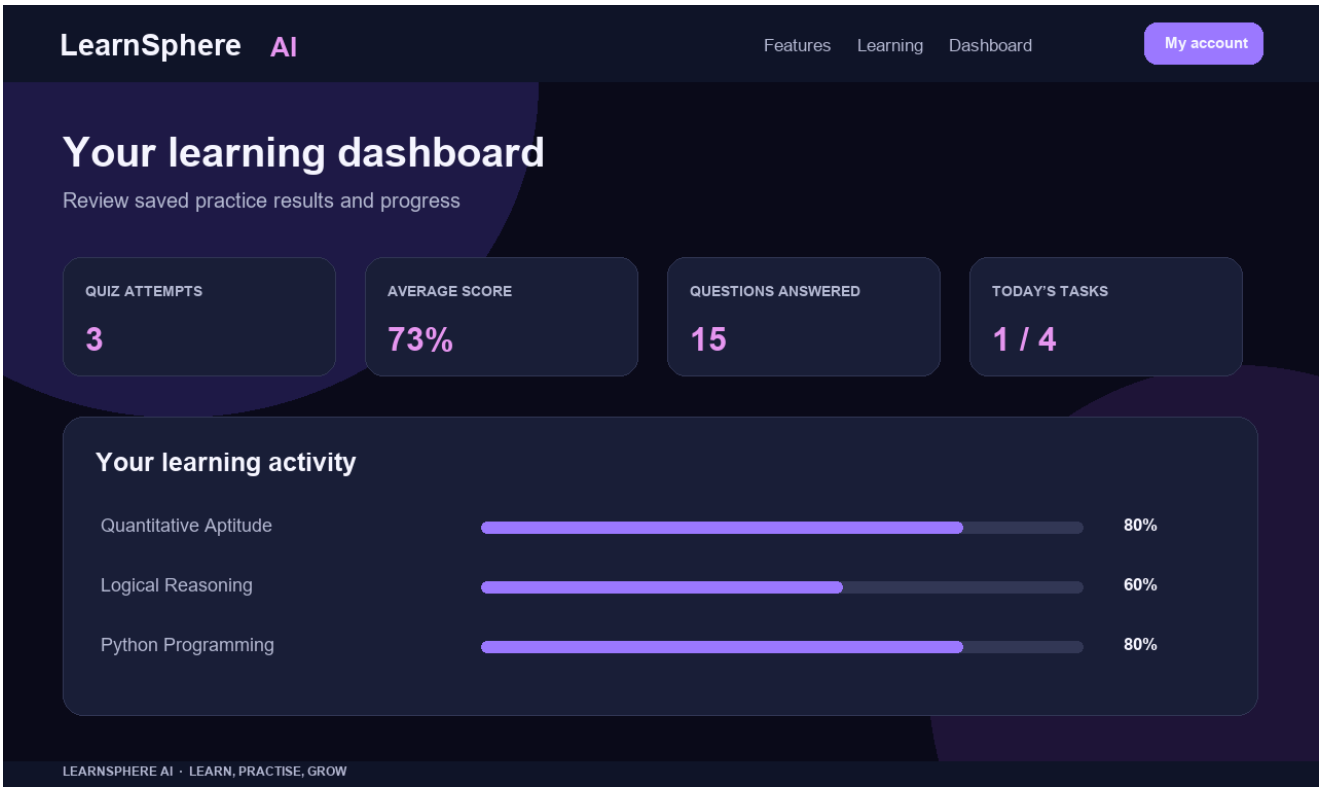
Learn. Practise. Build momentum.

LearnSphere AI brings placement preparation into one learner workspace. It combines short aptitude quizzes, coding practice, interview writing, English coaching, and AI study support. This guide explains what each area does and how the app records progress.

Live website

learnsphere-ai-ddv4.onrender.com

1 | Start with your learning home



Dashboard preview: saved quiz results, practice totals, and daily tasks at a glance.

The homepage is the entry point. Visitors can explore the learning areas and create an account. Signed-in learners see a daily progress card that updates from saved activity rather than fixed sample numbers. The main call to action takes an authenticated learner to their dashboard.

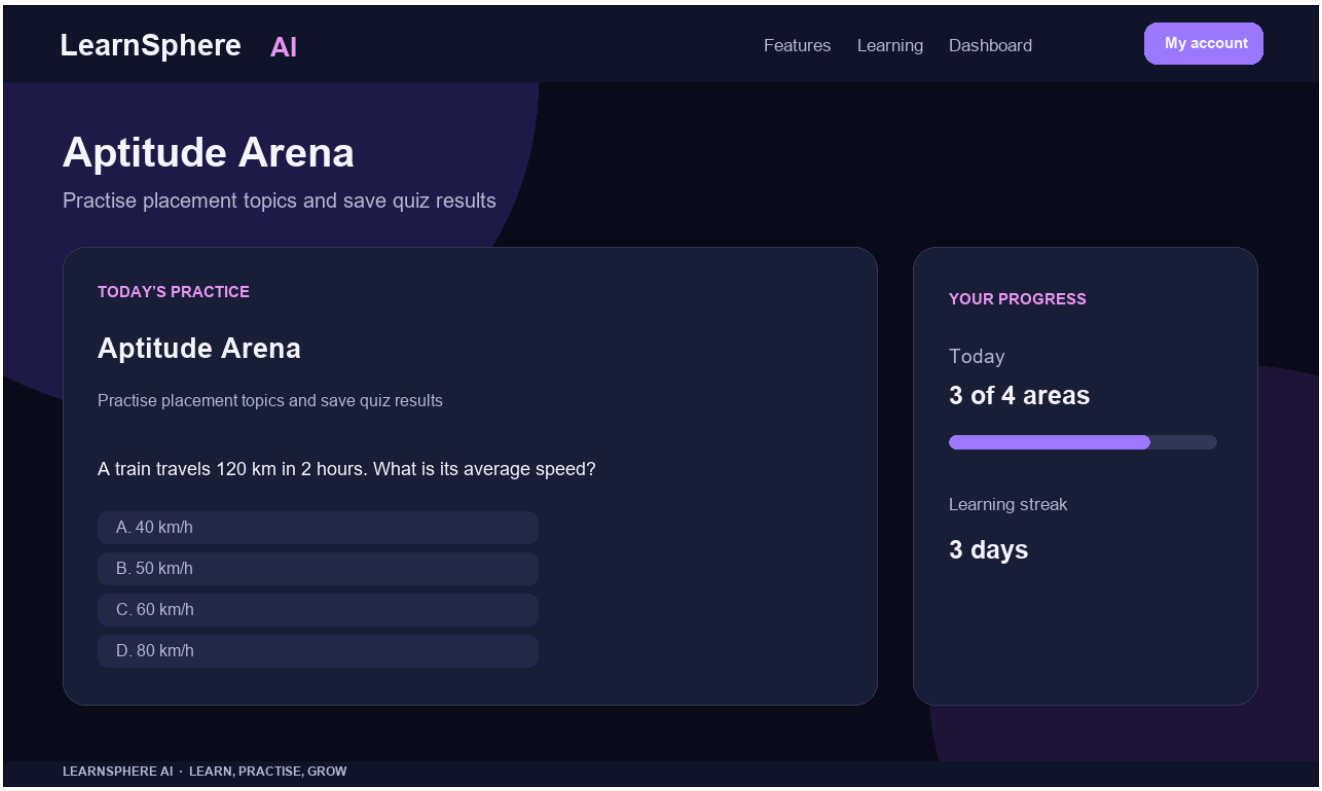
How today's progress works

The card tracks four focus areas: aptitude, coding, interview practice, and communication. Completing any activity in a focus area marks that area for the current date. The bar advances in 25-point steps: 0%, 25%, 50%, 75%, and 100%. Repeating the same area on the same day does not inflate the total.

The learning streak advances at most once per calendar day when an activity is recorded. Practising multiple areas in one day still counts as one streak day. If a learner misses a calendar day, the next recorded practice starts a new streak. Opening a page alone does not count as completed practice.

The progress record is per account. A new LearningActivity table stores the category and date; the existing user streak fields keep the consecutive-day count. The table is created by the app's existing SQLAlchemy create_all startup path.

2 | Practice areas

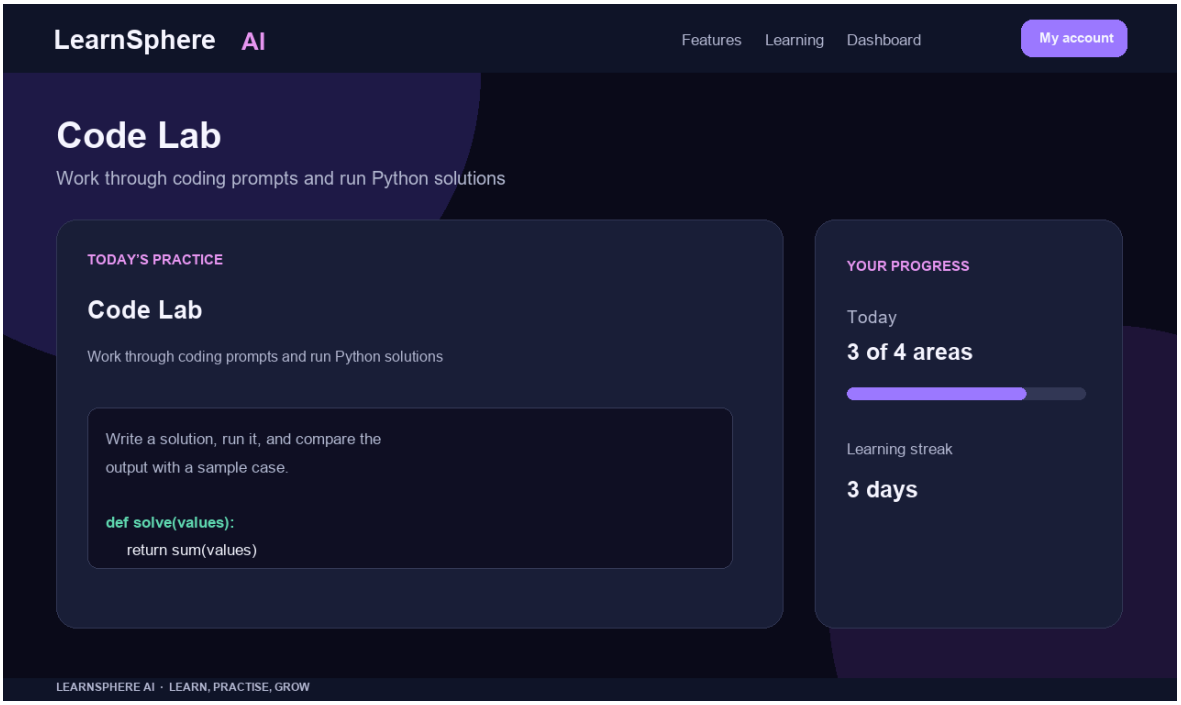


Aptitude Arena page preview. Quiz results are saved to the learner account.

Aptitude Arena

Short five-question quizzes cover quantitative aptitude, logical reasoning, Python, operating systems, DBMS, and computer networks. A valid saved result counts toward the aptitude focus area. Scores contribute to dashboard totals and topic averages.

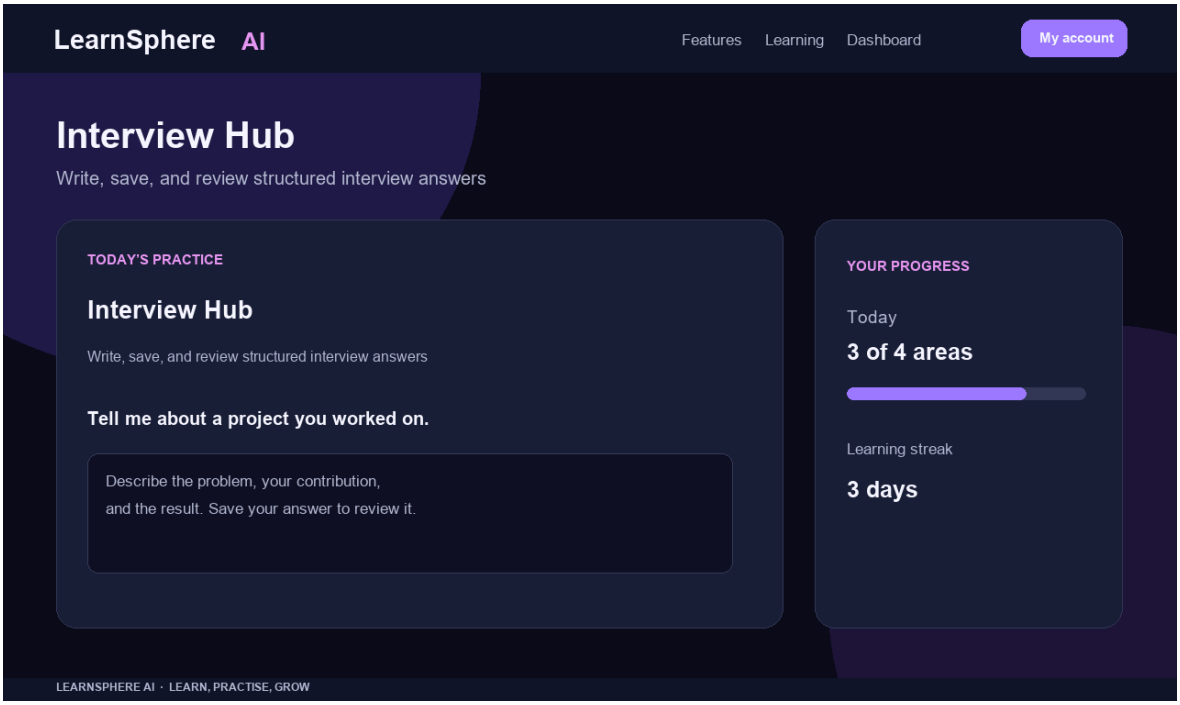
Code Lab



Code Lab page preview. Learners can practise a generated challenge and run Python code.

Code Lab presents coding challenges and supports running Python solutions. Completing a challenge records coding activity. The runner applies a time limit and input/output size limits; deployment should be treated as a learning demo, not a hardened public code execution service.

3 | Communication and interviews



Interview Hub page preview. A saved practice answer counts toward interview progress.

Interview Hub

Learners choose a prompt, write an answer, and get a lightweight review of structure, personal contribution, results, and reflection. Saving an answer records interview practice. The separate challenge track contains up to 100 levels and saves completed responses.

English Coach

Writing and pronunciation feedback are generated through the configured AI service. Successful feedback records communication practice. AI calls need a valid GEMINI_API_KEY in the app environment; the key belongs in Render environment variables and should never be placed in browser code or committed to Git.

AI Coach and Study Assistant

The AI Coach supports career planning; the Study Assistant explains topics and helps with study requests. Successful exchanges are saved to the learner's account and contribute to a learning streak, while the four focus-area bar remains tied to its four named categories.

Placement Plan

The daily plan suggests one task in each focus area. When a learner marks a plan task complete, the task is saved and its category contributes to today's progress.

4 | Accounts, data, and request flow

The app uses Flask routes and server-rendered Jinja templates. Flask-Login protects learner pages and associates saved activities with the authenticated user. Registration stores a Werkzeug password hash; login verifies the supplied password against that hash. The raw password is not stored.

Typical request flow

1. Browser	Learner submits a form or requests a page.
2. Flask route	Authentication is checked; inputs are validated and the feature action runs.
3. Services / models	AI services call Gemini when needed; SQLAlchemy saves results and activity.
4. Response	The route returns JSON or renders a Jinja page with account-specific data.

Main data models

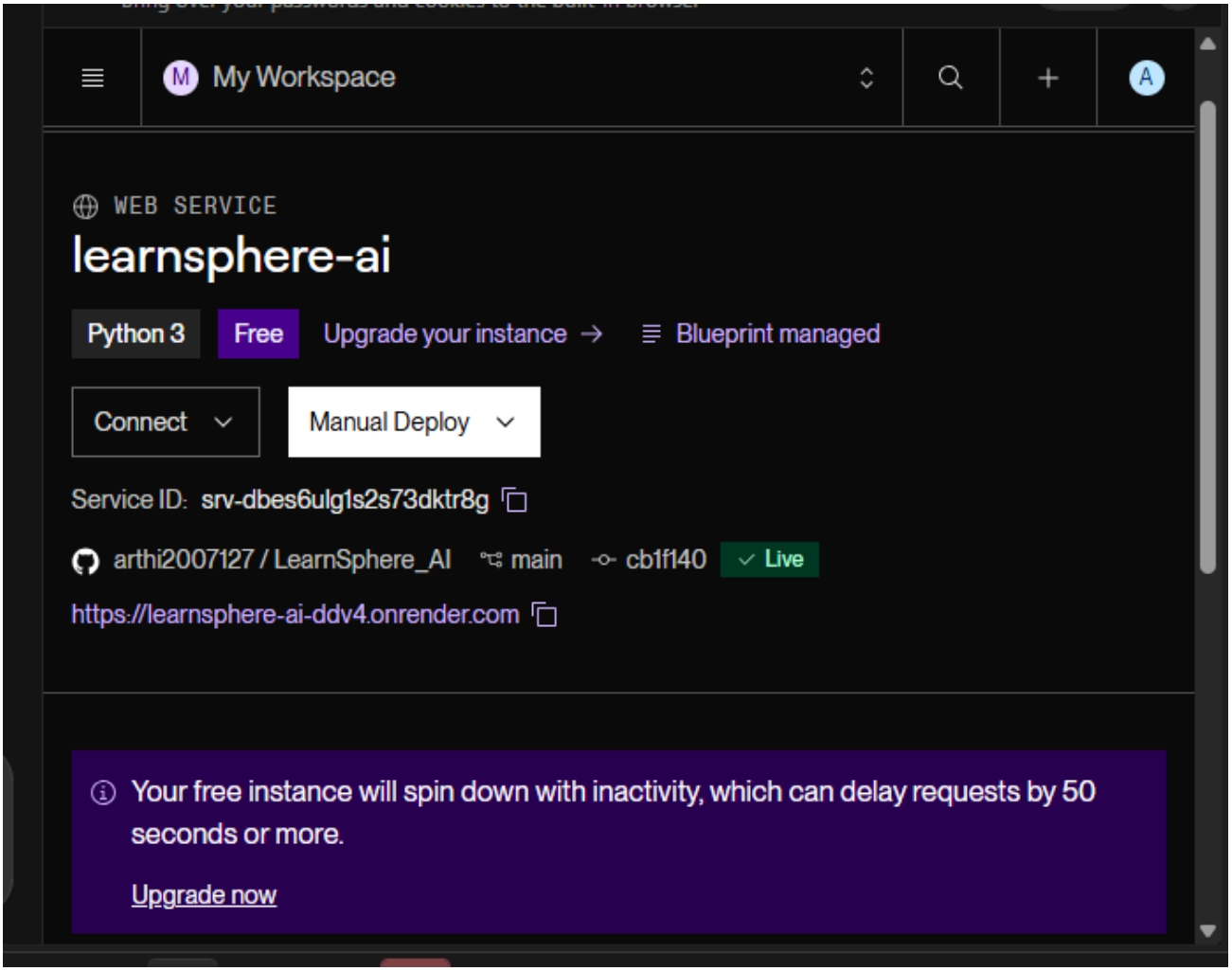
User holds account identity, password hash, role, and streak. **QuizAttempt** stores topic and score. **InterviewPracticeSession** stores prompt and written answer. **PlacementPlanCompletion** stores each daily task. **StudyAssistantMessage** stores AI conversation history. **LearningActivity** stores a once-per-day category record for the homepage progress card.

Credentials and tokens

The Gemini API key is read by server-side services from environment configuration. Password reset uses a signed, time-limited token; it is validated by the server and is not a reusable password. Session authentication is managed by Flask-Login and the Flask secret key. Configure SECRET_KEY, database credentials, SMTP credentials, and GEMINI_API_KEY as deployment secrets.

5 | Live deployment and next steps

The repository is connected to a Render web service and the dashboard shows the service as Live. Use the public URL above to open the app. The free web service may sleep after inactivity, so its first request can take longer while it wakes.



Render dashboard screenshot supplied by the project owner (25 October 2026). It shows the live service, connected GitHub repository, and public URL.

Deployment operations

Render service overview

Operational notes

• AI features require GEMINI_API_KEY to be present in the Render service environment.

• The project’s Render Blueprint uses free resources; its free PostgreSQL database has a limited lifetime. Do not rely on it for permanent learner records.

• Back up or migrate learner data before changing or replacing the production database.

• Before inviting untrusted users, strengthen code execution isolation and review rate limits, logging, account recovery, and privacy practices.

Project files

app.py contains routes and request handling. models.py defines persisted account and learning data. services/ contains AI-backed feature logic. templates/ contains the website pages, and static/ contains styles and browser assets. README.md documents setup and deployment.

Made for learning and growth. Visit the live LearnSphere AI website.